Embeddings Detail and RAG

We will begin 8:40 PM

Homework June 02, 2026

1. Design the tool-schema for Weather API
2. Function for talking to the API
3. Design the flow for your BE to talk to LLM, and send response of API back to LLM to get the fina
answer.

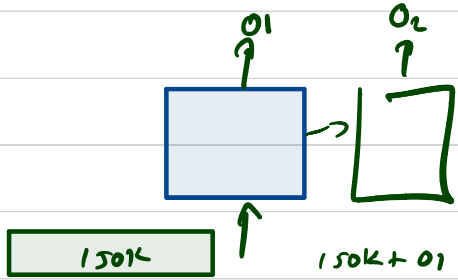
Why do we need RAG?

Close to 380K window → Use Larger model

→ Compact (Lossy compression)

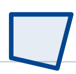
→ Remove First 50K window

```
Settings  Status  Config  Usage  Stats

Session

Total cost:            $0.0000
Total duration (API):  0s
Total duration (wall): 4s
Total code changes:    0 lines added, 0 lines removed
Usage:                 0 input, 0 output, 0 cache read, 0 cache write

Current session
                                                 22% used
Resets 11:10pm (Asia/Calcutta)

Current week (all models)
                                                 3% used
Resets Jun 6 at 10:30am (Asia/Calcutta)

What's contributing to your limits usage?
Approximate, based on local sessions on this machine — does not include
other devices or claude.ai

Last 7d · these are independent characteristics of your usage, not a
breakdown

97% of your usage came from sessions active for 8+ hours
 These are often background/loop sessions. Continuous usage can add up
 quickly so make sure it is intentional.

31% of your usage was at >150k context
 Longer sessions are more expensive even when cached. /compact mid-task,
 /clear when switching to new tasks.

Skills, subagents, plugins, and MCP servers
No attribution data yet · accumulates as you use Claude

d to day · w to week

Usage credits
                                                 32% used
$16.23 / $50.00 spent · Resets Jun 1 (Asia/Calcutta)

Esc to cancel
```



M_1 – 400K / 50K → a_1 (S_1)

M_1 → 400K / 350K → a_2 (S_2)

S_1  S_2

a. <

b. >

c. =

10 doc → Ans

100 doc →

Customer Service

50K - pdf files → query - "How do I print on transparent pixels - API"

Embedding model

Query → GPT → tokens / words

Query → Embeddin → Embedding

Step 1

Pdf $_1$ → Embedding Model → 1D - 768

Pdf $_2$ → Embedding Model → 1D - 768

Pdf $_3$ → Embedding Model → 1D - 768

⋮

Pdf $_{50K}$ → Embedding Model → 1D - 768

Step 2

Query

↓

Embedding Model

↓

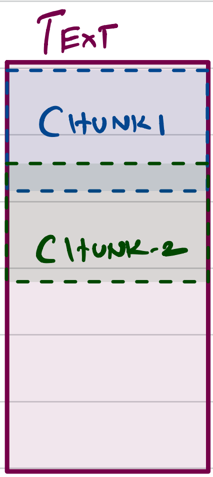
1D - 768

STEP -3

Cosine-Sim(E-P_1, Query)
Cosine-Sim(E-P_2, Query)
⋮
Cosine-Sim(E-P_{5014}, Query)

Sort on Score
↓
Top 5 most sim pdf
→ Text of 5 PdF
+
Query
↓
LLM → Answer
↓
User

PDF -1



0th – 300th words.
Name of the perp is....
250th – 550th words.
~~300th – 600th words~~

Gabbar singh, He is ....

Query ⇒ Relevant chunks — Chunk 1 + chunk 2
+ chunk 3

Why do we need FAISS ?

Embeddings → 1 M documents
10^6

Query Vector → Embedding.

Vanilla computation $= 10^6$

\# of Computations

$10 + 10 + 10K$
$= 10K$

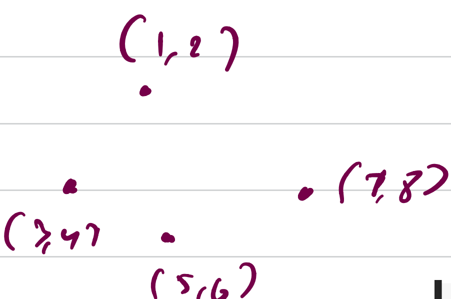
K_1 — 100K — SK_1 — 10K
SK_2 — 10K
⋮
SK_{10} — 10K

K_2 — 100K — SK_1 — 10K
SK_2 — 10K
⋮
SK_{10} — 10K

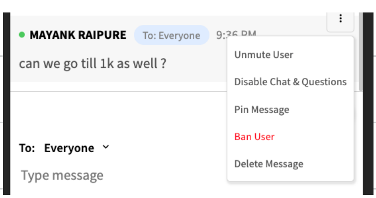
⋮

K_{10} — 100K — SK_1 — 10K
SK_2 — 10K
⋮
SK_{10} — 10K

Reduction $\frac{10^6}{10^3} = 1000$



$$= \left(\frac{1+3+5+7}{4}, \frac{2+4+6+8}{4} \right)$$



```
# Confidence/margin heuristic: how many top hits should we actually expand?
# - If the best match is strong (>= 0.35) AND clearly beats #2 (margin >= 0.05),
#   we trust just that one hit.
# - Otherwise the results look ambiguous, so cast a wider net (up to 3).
top1   = float(D[0][0])
top2   = float(D[0][1]) if len(D[0]) > 1 else 0.0
margin = top1 - top2
init_topk = 1 if (top1 >= 0.35 and margin >= 0.05) else min(3, topk)
```

Relevance Score for doc1 — 0.96
" " " doc2 — 0.23

0.73 ↑M